School of
Management and Law

MDM: Model Deployment & Maintenance
DJL Training & Inference

Adrian Moser
adrian.moser@zhaw.ch, FS2026

DJL Training

Training mit DeepJavaLibrary

Beispiel: Footwear Classification

Daten

UT Zappos50K

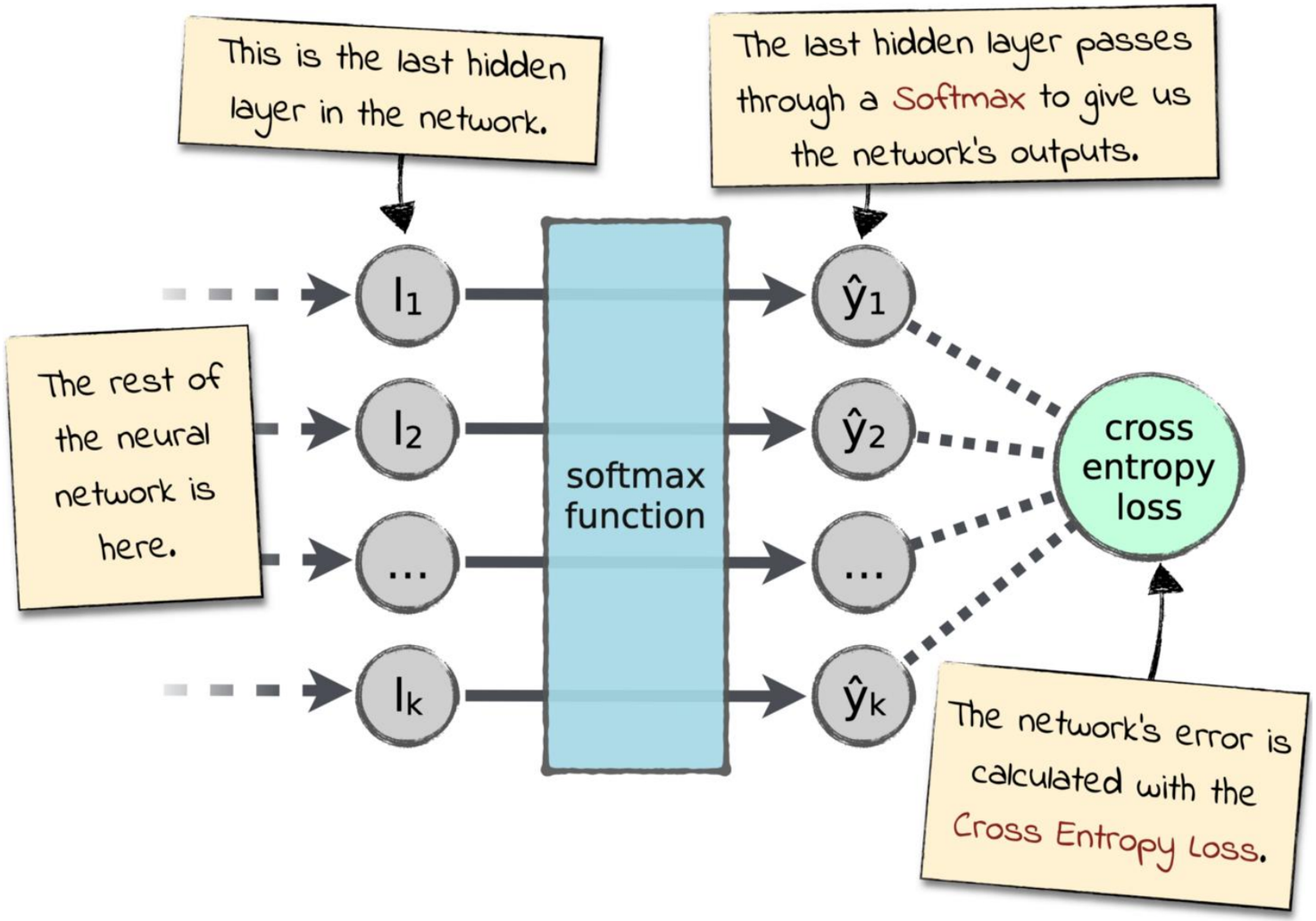
- Zur Verfügung gestellt von der University of Texas
- Ein Schuhdatensatz, der aus 50’025 Katalogbildern besteht
- Bilder von Zappos.com gesammelt

Bilder

- In 4 Hauptkategorien unterteilt – Schuhe, Sandalen, Hausschuhe und Stiefel
- weitere funktionalen Typen und einzelnen Marken
- weisser Hintergrund
- zentriert
- gleiche Ausrichtung

Repetition Begriffe Training

Begriff	Erläuterung
Training Dataset	Datenset für Training des Modells
Validation Dataset	Datenset für Validierung des Modells
Loss	Verlust ist eine Zahl, die angibt, wie schlecht die Vorhersage des Modells für ein einzelnes Beispiel war. Wenn die Vorhersage des Modells perfekt ist, ist der Verlust null. Andernfalls ist der Verlust grösser.
Accuracy	Anteil korrekter Vorhersagen: Die Anzahl korrekter Vorhersagen dividiert durch die Gesamtzahl der Vorhersagen.
Epoch	Ein vollständiger Trainingsdurchlauf über den gesamten Trainingssatz, so dass jedes Beispiel einmal verarbeitet wurde.
Loss function	Während des Trainings oder Tests eine mathematische Funktion, die den Verlust anhand einer Reihe von Beispielen berechnet. Eine Verlustfunktion gibt für Modelle mit guten Vorhersagen einen geringeren Verlust zurück als für Modelle mit schlechten Vorhersagen.


This is the last hidden layer in the network.
The last hidden layer passes through a Softmax to give us the network's outputs.
The rest of the neural network is here.
l_1
l_2
...
l_k
softmax function
$\hat{y}_1$
$\hat{y}_2$
...
$\hat{y}_k$
cross entropy loss
The network's error is calculated with the Cross Entropy Loss.

Training.java: Daten vorbereiten

Daten laden (Methode initDataset) und aufteilen

```
ImageFolder dataset = initDataset("ut-zap50k-images-square");
RandomAccessDataset[] datasets = dataset.randomSplit(8, 2);
```

initDataset

```
private static ImageFolder initDataset(String datasetRoot)
throws IOException, TranslateException {
        ImageFolder dataset = ImageFolder.builder()
                        // retrieve the data
                        .setRepositoryPath(Paths.get(datasetRoot))
                        .optMaxDepth(10)
                        .addTransform(new Resize(Models.IMAGE_WIDTH, Models.IMAGE_HEIGHT))
                        .addTransform(new ToTensor())
                        // random sampling; don't process the data in order
                        .setSampling(BATCH_SIZE, true)
                        .build();
        dataset.prepare();
        return dataset;
}
```

Models.java: Leeres Modell initialisieren

Statische Methode erzeugt «leeres» Modell und weist Neurales Netzwerk zu

ResNet: Deep Residual Learning for Image Recognition

```
public static final String MODEL_NAME = "shoeclassifier";

public static Model getModel() {
        // create new instance of an empty model
        Model model = Model.newInstance(MODEL_NAME);

        Block resNet50 =
                ResNetV1.builder() // construct the network
                        .setImageShape(new Shape(3, IMAGE_HEIGHT, IMAGE_WIDTH))
                        .setNumLayers(50)
                        .setOutSize(NUM_OF_OUTPUT)
                        .build();

        // set the neural network to the model
        model.setBlock(resNet50);
        return model;
}
```

Training.java: Konfiguration Training

Loss Function bereitgestellt in der Klasse Loss

Das Resultat ist eine TrainingConfig

```
Loss loss = Loss.softmaxCrossEntropyLoss();
TrainingConfig config = setupTrainingConfig(loss);
```

```
private static TrainingConfig setupTrainingConfig(Loss loss) {
        return new DefaultTrainingConfig(loss)
                .addEvaluator(new Accuracy())
                .addTrainingListeners(TrainingListener.Defaults.logging());
}
```

Training.java: Modell trainieren und speichern

Trainer erstellen mit TrainingConfig

Training mit EasyTrain und Datensätzen ausführen

```
Model model = Models.getModel(); // empty model instance to hold patterns
Trainer trainer = model.newTrainer(config);
trainer.setMetrics(new Metrics());

Shape inputShape = new Shape(1, 3, Models.IMAGE_HEIGHT, Models.IMAGE_HEIGHT);
trainer.initialize(inputShape);

EasyTrain.fit(trainer, EPOCHS, datasets[0], datasets[1]);

TrainingResult result = trainer.getTrainingResult();
model.setProperty("Epoch", String.valueOf(EPOCHS));
model.setProperty("Accuracy", String.format("%.5f", result.getValidateEvaluation("Accuracy")));
model.setProperty("Loss", String.format("%.5f", result.getValidateLoss()));

model.save(modelDir, Models.MODEL_NAME);
Models.saveSynset(modelDir, dataset.getSynset());
```

Auszüge Training-Log

Bilder laden

```
15:11:36.508 [main] DEBUG ai.djl.basicdataset.cv.classification.AbstractImageFolder - Loaded 12834 images in C:\mosa\dev\edu\3MDM\djl-footwear-classification\djl-
footwear_classification\ut-zap50k-images-square\Boots, class: 0
15:11:37.023 [main] DEBUG ai.djl.basicdataset.cv.classification.AbstractImageFolder - Loaded 18575 images in C:\mosa\dev\edu\3MDM\djl-footwear-classification\djl-
footwear_classification\ut-zap50k-images-square\Sandals, class: 1
15:11:39.719 [main] DEBUG ai.djl.basicdataset.cv.classification.AbstractImageFolder - Loaded 48783 images in C:\mosa\dev\edu\3MDM\djl-footwear-classification\djl-
footwear_classification\ut-zap50k-images-square\Shoes, class: 2
15:11:39.820 [main] DEBUG ai.djl.basicdataset.cv.classification.AbstractImageFolder - Loaded 50066 images in C:\mosa\dev\edu\3MDM\djl-footwear-classification\djl-
footwear_classification\ut-zap50k-images-square\Slippers, class: 3
\djl-footwear_classification>
```

PyTorch Engine wird geladen

```
15:11:39.844 [main] DEBUG ai.djl.engine.Engine - Registering EngineProvider: PyTorch
15:11:39.846 [main] DEBUG ai.djl.engine.Engine - Found default engine: PyTorch
15:11:39.877 [main] DEBUG ai.djl.util.cuda.CudaUtils - No cudart library found in path.
15:11:39.899 [main] DEBUG ai.djl.pytorch.jni.LibUtils - Using cache dir: C:\Users\mosa\.djl.ai\pytorch\1.12.1-cpu-win-x86_64
15:11:39.907 [main] DEBUG ai.djl.pytorch.jni.LibUtils - Loading native library: C:\Users\mosa\.djl.ai\pytorch\1.12.1-cpu-win-x8664\asmjit.dll
15:11:39.936 [main] DEBUG ai.djl.pytorch.jni.LibUtils - Loading native library: C:\Users\mosa\.djl.ai\pytorch\1.12.1-cpu-win-x8664\c10.dll
15:11:39.971 [main] DEBUG ai.djl.pytorch.jni.LibUtils - Loading native library: C:\Users\mosa\.djl.ai\pytorch\1.12.1-cpu-win-x8664\libiomp5md.dll
15:11:40.002 [main] DEBUG ai.djl.pytorch.jni.LibUtils - Loading native library: C:\Users\mosa\.djl.ai\pytorch\1.12.1-cpu-win-x8664\uv.dll
64\uv.dll
15:11:40.092 [main] DEBUG ai.djl.pytorch.jni.LibUtils - Loading native library: C:\Users\mosa\.djl.ai\pytorch\1.12.1-cpu-win-x86_4\fbgemm.dll
64\fbgemm.dll
15:11:40.903 [main] DEBUG ai.djl.pytorch.jni.LibUtils - Loading native library: C:\Users\mosa\.djl.ai\pytorch\1.12.1-cpu-win-x86_5:11:41.341 [main] DEBUG
ai.djl.pytorch.jni.LibUtils - Loading native library: C:\Users\mosa\.djl.ai\pytorch\1.12.1-cpu-win-x8615:11:40.092 [main] DEBUG ai.djl.pytorch.jni.LibUtils - Loading native
library: C:\Users\mosa\.djl.ai\pytorch\1.12.1-cpu-win-x86_64\fbgemm.dll
15:11:40.903 [main] DEBUG ai.djl.pytorch.jni.LibUtils - Loading native library: C:\Users\mosa\.djl.ai\pytorch\1.12.1-cpu-win-x86_64\torch_cpu.dll
15:11:41.341 [main] DEBUG ai.djl.pytorch.jni.LibUtils - Loading native library: C:\Users\mosa\.djl.ai\pytorch\1.12.1-cpu-win-x86_64\torch.dll
15:11:41.368 [main] DEBUG ai.djl.pytorch.jni.LibUtils - Loading native library: C:\Users\mosa\.djl.ai\pytorch\1.12.1-cpu-win-x86_64\0.19.0-djl_torch.dll
…
```

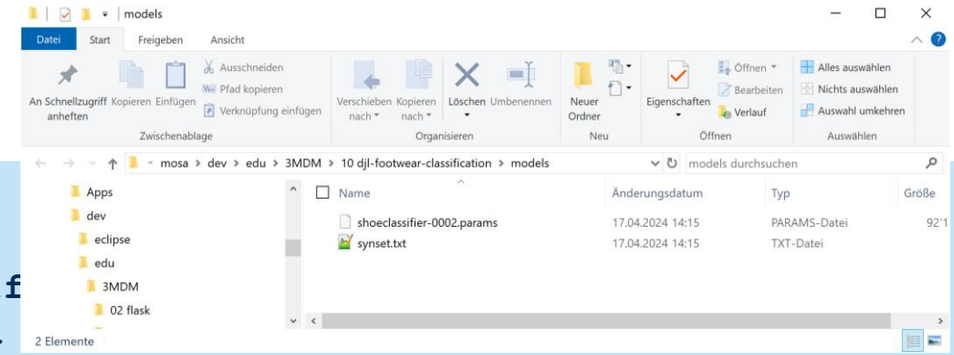
Auszüge Training-Log

Training wird durchgeführt

```
15:11:41.562 [main] INFO ai.djl.training.listener.LoggingTrainingListener - Training on: cpu().
15:11:41.564 [main] INFO ai.djl.training.listener.LoggingTrainingListener - Load PyTorch Engine Version 1.12.1 in 0.083 ms.
Training:    100% |========================================| Accuracy: 0.88, SoftmaxCrossEntropyLoss: 0.41
Validating:  100% |========================================|
16:36:10.435 [main] INFO ai.djl.training.listener.LoggingTrainingListener - Epoch 1 finished.
16:36:10.437 [main] INFO ai.djl.training.listener.LoggingTrainingListener - Train: Accuracy: 0.88, SoftmaxCrossEntropyLoss: 0.4116:36:10.440 [main] INFO
ai.djl.training.listener.LoggingTrainingListener - Validate: Accuracy: 0.91, SoftmaxCrossEntropyLoss: 0.27
Training:    100% |========================================| Accuracy: 0.92, SoftmaxCrossEntropyLoss: 0.24
Validating:  100% |========================================|
17:59:52.353 [main] INFO ai.djl.training.listener.LoggingTrainingListener - Epoch 2 finished.
17:59:52.356 [main] INFO ai.djl.training.listener.LoggingTrainingListener - Train: Accuracy: 0.92, SoftmaxCrossEntropyLoss: 0.24
17:59:52.357 [main] INFO ai.djl.training.listener.LoggingTrainingListener - Validate: Accuracy: 0.88, SoftmaxCrossEntropyLoss: 0.3817:59:52.866 [main] INFO
ai.djl.training.listener.LoggingTrainingListener - forward P50: 900.111 ms, P90: 1118.603 ms
17:59:52.883 [main] INFO ai.djl.training.listener.LoggingTrainingListener - training-metrics P50: 0.015 ms, P90: 0.021 ms
17:59:52.899 [main] INFO ai.djl.training.listener.LoggingTrainingListener - backward P50: 1957.840 ms, P90: 2382.778 ms
17:59:52.917 [main] INFO ai.djl.training.listener.LoggingTrainingListener - step P50: 231.689 ms, P90: 294.855 ms
17:59:52.918 [main] INFO ai.djl.training.listener.LoggingTrainingListener - epoch P50: 5068.859 s, P90: 5068.859 s
PS C:\mosa\dev\edu\3MDM\djl-footwear-classification\djl-footwear_classification>
```

Modell und Gruppierung erstellt

```
Mode                LastWriteTime         Length Name
----                -------------         ------ ----
-a----        15.11.2022     17:59       94387051 shoeclassif
-a----        15.11.2022     17:59             28 synset.txt
```



DJL Inference und UI

Inference.java: Modell laden

Wir laden nun unser eigenes Modell:

```
Model model = Models.getModel();
Path modelDir = Paths.get("models");
model.load(modelDir, Models.MODEL_NAME);
```

Vorbedingung:

Modell wurde erstellt, trainiert und im Verzeichnis «models» gespeichert.

Inference.java: Predictor erstellen

Aus dem Modell wird ein Predictor erstellt, der zur Inference verwendet werden kann.
Der Predictor enthält auch einen Translator.

```
Predictor<Image, Classifications> predictor;

…

Translator<Image, Classifications> translator = ImageClassificationTranslator.builder()
                    .addTransform(new Resize(Models.IMAGE_WIDTH, Models.IMAGE_HEIGHT))
                    .addTransform(new ToTensor())
                    .optApplySoftmax(true)
                    .build();
predictor = model.newPredictor(translator);
```

Translator

- Bild in die richtige Grösse bringen
- Umwandlung in Tensor
- Anwendung Softmax auf Output (nur wenn nicht bereits in neuralem Netz enthalten)

Inference.java: Predict-Methode

Umwandlung des Bildes aus byte[ ] in ein DJL Image.

Aufruf des Predictors und Rückgabe der Classifications.

```
public Classifications predict(byte[] image) throws ModelException, TranslateException, IOException {
            InputStream is = new ByteArrayInputStream(image);
            BufferedImage bi = ImageIO.read(is);
            Image img = ImageFactory.getInstance().fromImage(bi);

            Classifications predictResult = this.predictor.predict(img);
            return predictResult;
}
```

Bild-Umwandlung

- Eingabe als Byte-Array byte[ ]
- Lesen des Array mit einem InputStream
- Umwandlung in BufferedImage (Subtyp von java.awt.Image)
- Predict-Methode verlangt Image als Input

ClassificationController.java: REST Service

Einmalige Erstellung des Objekts Inference (Controller ist Singleton).

Aufruf von «predict» für jeden POST-Request auf /analyze.

Rückgabe des Resultats als JSON.

```
@PostMapping(path = "/analyze")
public String predict(@RequestParam("image") MultipartFile image) throws Exception {
        return inference.predict(image.getBytes()).toJson();
}
```

Bild als Parameter

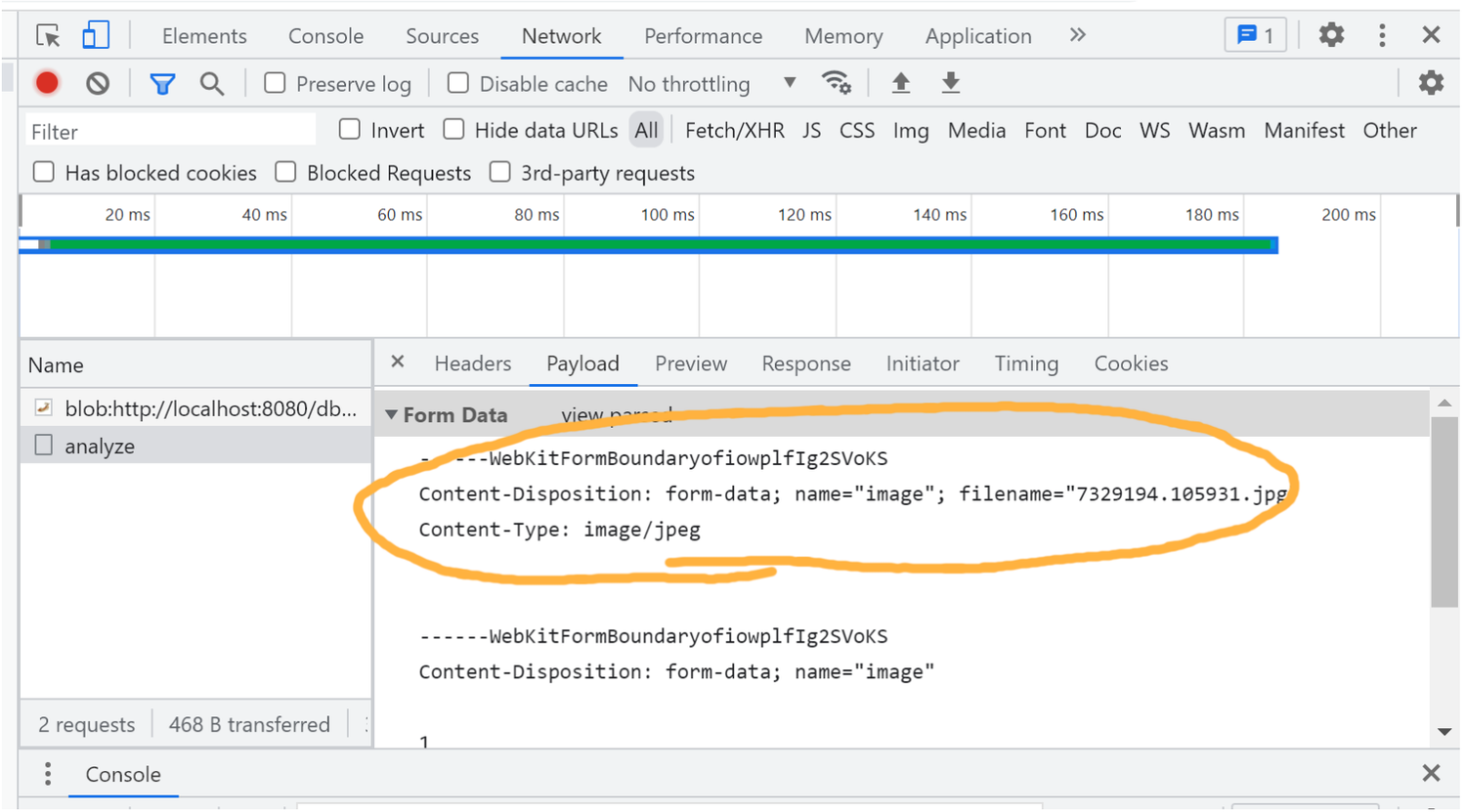
- POST-Request da Datei übermittelt wird
- Body enthält Multipart-File mit Name «image»

Aufruf des Classification Service

Mit Postman und Browser UI

Übermittlung Bild von Web-Browser an Backend

Analyse mit Developer-Tools / Inspektor (Ctrl-Shift-I in den meisten Browsern)



Aufruf des Service mit Postman

Postman

- Simulation von Backend-Requests
- Im Gegensatz zu Browser-URL-Feld auch komplexe Aufrufe wie POST-Requests

Vorbedingungen

- Registration bei https://www.postman.com/
- Installation Postman VS Code Extension

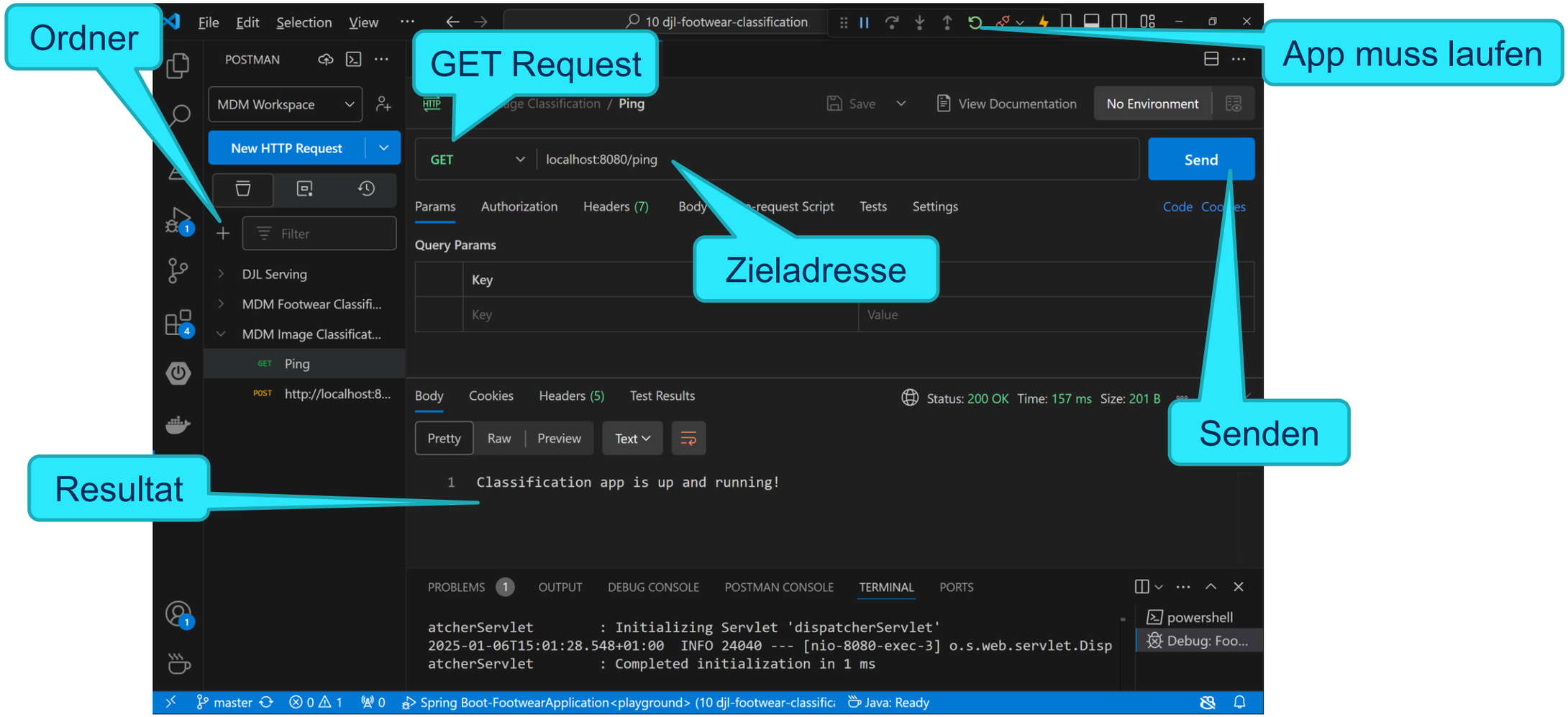
Alternativ: Postman Desktop Agent

- Postman läuft im Browser
- Postman Desktop Agent ermöglicht Zugriff auf localhost

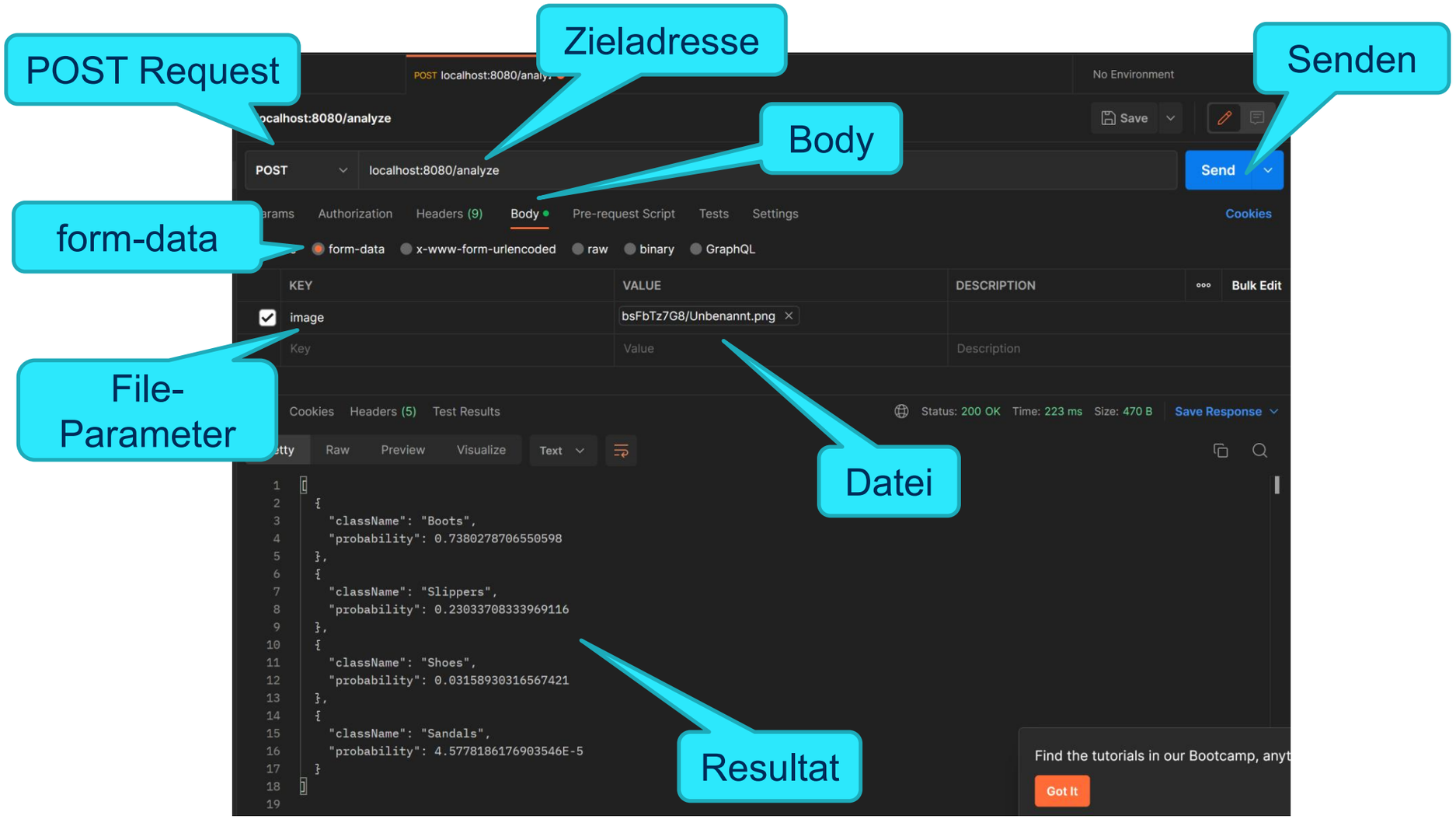
Alternativ: Postman App

- Postman App verwenden

Postman Hello World mit GET-Request (mit VS Code Extension)



POST-Request mit Bild (mit Postman Web App)



UI

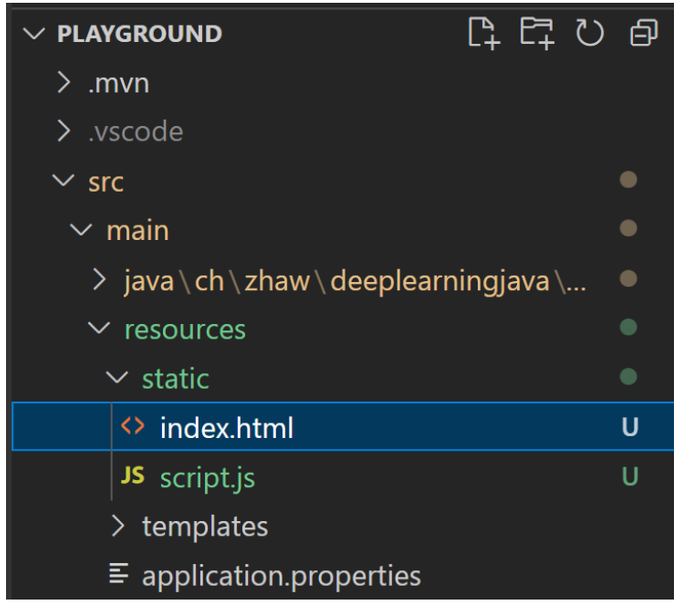
Ergänzung mit einfachem UI

Konzept bekannt aus Python-Beispiel bzw. DJL-Beispiel

Static Resources in Spring Boot (Repetition)

src/main/resources/stats

- Spring Boot zeigt Dateien aus diesem Verzeichnis automatisch an
- index.html als Einstiegsseite
- script.js für Aufruf REST-Service



Beispiel index.html

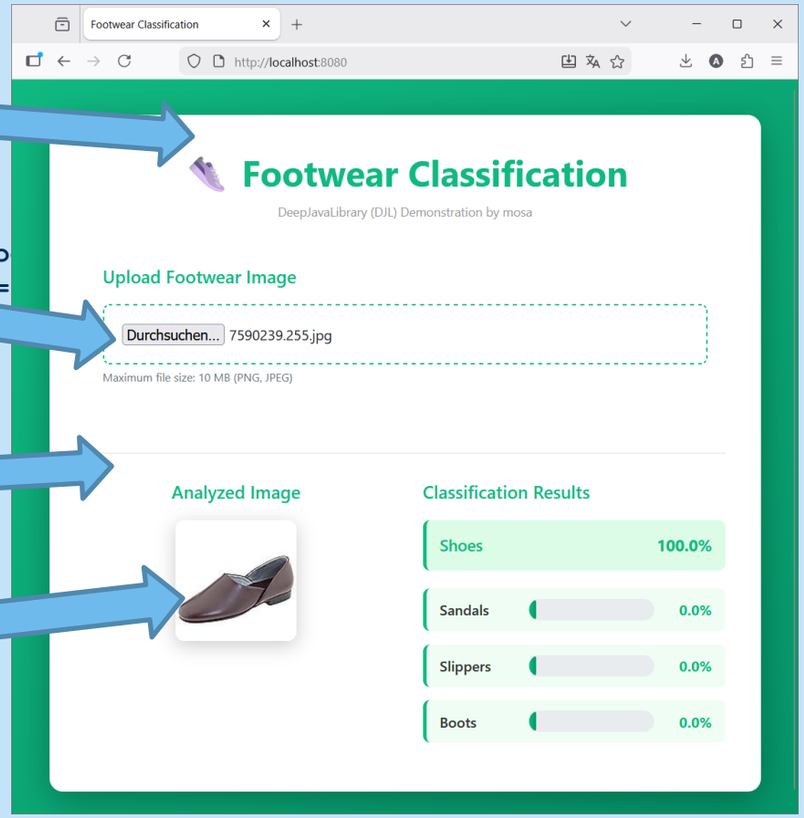
Vergleich HTML (vereinfacht) und UI, JavaScript-Aufruf «onchange»

```
<body>
    <div class="container-fluid">
        <h1>Image Classification</h1>
        <p>DLJ Demonstration by mosa</p>

        <form>
            <div class="form-group">
                <label for="exampleFormControlFile1">Bitte Bild hochladen (jpeg, png)</lab
                <input type="file" accept="..." class="form-control-file" name=
                    onchange="checkFiles(this.files);">
            </div>
        </form>

        <div id="answerPart" style="visibility: hidden;">
            <p><b>Answer:</b>
            <div id="answer"></div>
            </p>

            <p id="previewPart"><b>Analyzed Image:</b>
            <p><img id="preview" width="300" src="#" alt="preview" /></p>
            </p>
        </div>
    </div>
</body>
```



JavaScript: Bild mit POST

```
    // Preview
    if (file) {
        preview.src = URL.createObjectURL(files[0])
    }

    // Upload
    const formData = new FormData();
    for (const name in files) {
        formData.append("image", files[name]);
    }

    fetch('/analyze', {
        method: 'POST',
        headers: {
        },
        body: formData
    }).then(
        response => {
            console.log(response)
            response.text().then(function (text) {
                answer.innerHTML = text;
            });

        }
    ).then(
        success => console.log(success)
    ).catch(
        error => console.log(error)
    );
}
```

Bild lokal anzeigen

Gleicher Parametername!

Fetch API POST

Text zuweisen

Weiterführende Informationen

Vorlesungsbeispiel:
https://github.com/mosazhaw/djl-footwear_classification

Blog-Post:
https://towardsdatascience.com/deep-java-library-djl-a-deep-learning-toolkit-for-java-developers-55d5a45bca7e

Original Source-Code:
https://github.com/deepjavalibrary/djl-demo/tree/master/footwear_classification

Datenset:
https://vision.cs.utexas.edu/projects/finegrained/utzap50k/